

Devops – TP initiation

I. Préparation du projet

1. Installez node JS si ce n'est pas déjà fait
2. Dézippez le dossier disponible [ici](#) et ouvrez le dans une console
3. Exécutez "npm install"
4. Exécutez "npm test", tous les tests unitaires devraient fonctionner

II. Ajout à github

1. Créez un compte Github si vous n'en avez pas un
2. Créez un repository sur votre github
3. Ajoutez le projet à votre compte github (les instructions sont notées sur votre nouveau repository ("...or push an existing repository from the command line"))

III. Github actions pour vérifier les tests

1. Allez sur le tab actions de votre repository
2. Cliquez sur "Set up a workflow yourself"
3. Utilisez ce [lien](#) pour mettre en place un testing automatique de votre repository
4. Commit puis push de vos changements.

Vous pouvez voir l'état du build dans le tab actions après avoir commit & push.

IV. Mise en place de git flow

1. Install git flow (instructions [ici](#))
2. Initialisez votre repository avec git flow en exécutant « git flow init »
3. Gardez la configuration de base de git flow.

Vous devriez maintenant être sur develop.

Rappel :

-Develop = staging dans notre exemple

-Master/main = production

Dans le lien partagé précédemment, vous trouverez toutes les commandes utiles à git flow (notamment les releases)

V. Automatisation de mise en production/staging

1. Automatisez la création d'une image docker via une github actions en suivant les instructions sur ce [lien](#), celle-ci aura pour tag xxxx:**staging** (au lieu de latest) et le build se lancera lorsqu'un commit est fait sur la branche develop.
Attention: l'espacement du fichier yaml est mauvais et les liens sont à changer pour correspondre à votre github "deselikem" notamment. Une des commandes docker de l'exemple est à enlever, comme vu dans le cours
Le Dockerfile est aussi à retravailler comme expliqué dans le cours.
Il faudra aussi donner l'accès en écriture au container registry de github dans la github action. [Voir ici](#)

2. Créez un nouveau compte sur render.com.

Render.com est un service d'hosting comme un autre, qui propose un plan gratuit, ce qui nous est utile dans ce cas

3. Créez un nouveau service web à partir d'une image existante, l'url de l'image docker est disponible
4. Collez l'url du container registry de votre app (pour la récupérer, allez à la racine de votre repo github et allez dans packages en bas à droite) ayant pour nom xxx-staging.
5. Créez une seconde github actions qui sera démarrée lorsqu'un push est fait sur la branche main et qui construira l'image docker mais cette fois ci avec un tag xxxx:production
6. Créez un second service web pour la production
7. Récupérez l'url du container registry, mettez la dans le second service web.
8. Faites une release avec git flow (explication ci-dessous)
 - git flow release start VOTRENOMDERELEASE
 - git flow release finish VOTRENOMDERELEASE
 - Sauvegardez le premier fichier text (nom de commit de merge depuis votre branch release sur master)
 - Tapez votre tag de release (MAJOR.MINOR.BUILD) et un petit message en dessous selon votre envie (en général ce sont les releases notes avec breaking changes) puis sauvegardez
 - Sauvegardez ce dernier fichier text (nom de commit de votre release vers develop)
 - Allez sur develop (git checkout develop), git push
 - Allez sur master, push
 - Faites un git push --tags pour pousser votre release

Vérifiez que tout se build correctement

VI. Outil de suivi de qualité (code coverage)

1. Créez un compte sur coveralls.io
2. Ajoutez votre repository
3. Modifiez votre github action qui run les npm tests pour faire fonctionner coveralls comme expliqué [ici](#).

Le code coverage est un indicateur intéressant de qualité de code. Il permet de connaître toutes les lignes que vos tests unitaires n'ont pas parcourues.